

Exemplo 1 Slide 05:

```
#include <iostream>
```

```
#include <cstdio>
```

```
#include <cstring>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    int x = 2;
```

```
    char str1[] = "abcd";
```

```
    char str2[] = "ola!";
```

```
    printf("x:\n Endereco: %d\n Tamanho: %d\n Conteudo: %d\n\n",  
          &x, sizeof(x), x);
```

```
    printf("str1:\n Endereco: %d\n Tamanho: %d\n Conteudo: %s\n\n",  
          str1, sizeof(str1), str1);
```

```
    printf("str2:\n Endereco: %d\n Tamanho: %d\n Conteudo: %s\n\n",  
          str2, sizeof(str2), str2);
```

```
    printf("str2:\n Endereco: %d\n Tamanho: %d\n Conteudo: %s\n\n",  
          &str2[0], sizeof(char) * (strlen(str2) + 1), str2);
```

```
    return 0;
```

```
}
```

```
#include <iostream>
```

```
#include <cstdio>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    int x = 2;
```

```
    int *y = &x;
```

```
    printf("x:\n Endereco: %d\n Tamanho: %d\n Conteudo: %d\n\n",
```

```
           &x, sizeof(x), x);
```

```
    printf("y:\n Endereco: %d\n Tamanho: %d\n Conteudo: %d\n Conteudo apontado: %d\n\n",
```

```
           &y, sizeof(y), y, *y);
```

```
    return 0;
```

```
}
```

Slide 11:

```
#include <iostream>
```

```
using namespace std;
```

```
void somaprod(int a, int b, int* p, int* q)
```

```
{
```

```
    *p = a + b;
```

```
    *q = a * b;
```

```
}
```

```
int main()
```

```
{
```

```
    int s, p;
```

```
    somaprod(3, 5, &s, &p);
```

```
cout << "Soma = " << s << " e Produto = " << p << endl;
```

```
return 0;
```

```
}
```

O que é um ponteiro

Um **ponteiro** é uma variável que **guarda o endereço de outra variável**, não o valor diretamente.

Analogia do dia a dia

Imagine:

- Uma variável normal → é como **uma caixa com um objeto dentro**
- Um ponteiro → é como **um papel com o endereço dessa caixa**

👉 Ou seja:

- Valor direto = você tem a caixa
- Ponteiro = você sabe onde a caixa está

Por que usar ponteiros?

1. Alterar várias variáveis dentro de uma função

Problema (sem ponteiro)

Se você passa valores normais, a função recebe **cópias**.

```
void soma(int a)
```

```
{
```

```
    a = a + 10;
```

```
}
```

👉 O valor original **NÃO** muda.

Solução (com ponteiro)

```
void soma(int* a)
```

```
{
```

```
    *a = *a + 10;
```

}

👉 Agora você altera o valor original.

Exemplo do dia a dia

Imagine:

- Você dá uma **cópia de um documento** para alguém → mudanças não afetam o original
- Você dá o **acesso ao documento original** → qualquer mudança é real

👉 Ponteiro = acesso direto ao original

2. 🔄 Retornar mais de um valor

Funções normalmente retornam **apenas 1 valor**.

Sem ponteiro → limitado

int soma(int a, int b)

```
{  
    return a + b;  
}
```

Com ponteiro → vários resultados

void somaprod(int a, int b, int* soma, int* prod)

```
{  
    *soma = a + b;  
    *prod = a * b;  
}
```

Exemplo real

Você vai ao médico:

- Ele te entrega **um único resultado** (sem ponteiro) ❌
- Ou te entrega **vários exames ao mesmo tempo** (com ponteiro) ✅

👉 Ponteiros permitem isso!

3. ⚡ Economia de memória e desempenho

Sem ponteiro:

- Você copia dados grandes → lento

Com ponteiro:

- Você passa só o endereço → rápido

Exemplo prático

Imagine:

- Enviar um **vídeo de 2GB pelo WhatsApp** ❌ (lento)
- Enviar o **link do vídeo no Google Drive** ✅ (rápido)

👉 Ponteiro = enviar o “link” (endereço)

4. Estruturas dinâmicas

Ponteiros são essenciais para:

- Listas encadeadas
- Árvores
- Filas
- Alocação dinâmica (new, malloc)

Exemplo do dia a dia

Uma lista encadeada é como:

- Uma fila de pessoas onde cada pessoa **aponta para a próxima**

👉 Isso só funciona com ponteiros.

5. Comunicação direta com memória

Às vezes você precisa:

- Manipular hardware
- Trabalhar com sistemas
- Otimizar desempenho

👉 Ponteiros permitem acesso direto à memória.

Resumo simples

Use ponteiros quando você quer:

- ✓ Alterar valores fora da função
- ✓ Retornar vários resultados
- ✓ Evitar cópias grandes
- ✓ Criar estruturas complexas
- ✓ Trabalhar com memória diretamente

Ligando com seu código

No seu exemplo:

```
somaprod(3, 5, &s, &p);
```

👉 Você está dizendo:

“Função, vá diretamente nas variáveis `s` e `p` e coloque os resultados lá.”

Dica importante

Em C++, muitas vezes usamos **referências** no lugar de ponteiros:

```
void somaprod(int a, int b, int& s, int& p)
```

👉 Mais seguro e mais simples.